

CPS506 Lab 7

Rust: Functions, ownership, strong typing

Preamble

In this lab you will experiment with writing some simple Rust functions. These would be CPS109-level problems in Python but passing arguments and returning values in Rust becomes a lot trickier due to issues of ownership and strict typing.

Lab Description

Implement each of the following Rust functions in the single file called **main.rs**. This file is provided in the src directory of the provided Cargo project. Cargo projects in Rust are just like Mix projects in Elixir and Cabal projects in Haskell. You must work within the types that they indicate. If this requires you to perform casting, then so be it. Test your code by running 'cargo test' in the lab7 directory in a terminal.

1) **fn count_peaks(items: &[i32]) -> u32**

Given a reference to an input array, this function should count the number of “peaks” in the array. A peak is defined as an element flanked on either side by elements smaller than itself. The first and last elements of an array can still be considered peaks, if the single element adjacent to it is smaller.

2) **fn remove_runs(items: &[i32]) -> Vec<i64>**

This function accepts a reference to an array of integers as input and returns a vector where runs of identical integer values have been reduced to a single instance of that integer. For example, for the input array:

[1, 1, 2, 4, 5, 5, 5, 5, 7, 9, 9, 1, 1]

The remove_runs function would return the Vector:

< 1, 2, 4, 5, 7, 9, 1 >

Notice that we are not simply removing all duplicate elements! If there are two runs of 1, then each of those runs should be distilled to a single 1. We may or may not cover Vectors in class, but they are comparable to any other Vector type in any other language. Check out the documentation for information on using them:

<https://doc.rust-lang.org/std/vec/struct.Vec.html>

3) **fn count_and_remove_primes(items: &mut [u32]) -> u32**

This function accepts a mutable reference to an array. It should iterate through the array and determine if each number is prime. If the number is prime, set that element to 0. Return the count of primes that were found and set to zero in this way.

4) **fn safe_squares_rooks(n: u8, rooks: &[(u8, u8)]) -> u32**

In the game of chess, a rook can move anywhere along its row or column.

Given an n-by-n chessboard (parameter **n**) and an array of tuples representing row and column positions of the rooks (parameter **rooks**), this function must return the number of squares on the board that are safe from the rooks.

Note 1: Row and column indexes for the rooks start at 0. If the chessboard was 8x8, then valid row and column indexes would be 0 to 7.

Note 2: You may assume the row and column indexes of each rook are legal given the size of the chessboard. For example, if the chessboard is 8x8, no rook's row or column index will be off the board (>7).

Submission

Labs are to be completed and submitted *individually*. Submit your **main.rs** source file containing all completed functions on D2L, under the submission for Lab 7.